SpringForge

Intelligent Spring Boot Development Ecosystem

Spring Boot Development comes with challenges

late Code

Problem Statement & Stakeholders

Obeserved

Developers waste 50-70% of time on manual architecture setup, boilerplate coding, runtime error debugging, quality checks, and CI/CD configuration instead of building features.

Stakeholders

Spring Boot developers (primary), DevOps engineers and team leads (secondary), Engineering managers and CTOs (decision makers)

Context

Enterprise Java / Spring Boot development teams using IntelliJ IDEA

Why it

Lost development velocity delays product delivery, manual CI/CD configuration introduces pipeline errors and security misconfigurations, inconsistent code quality increases technical debt, and repeated boilerplate work raises engineering costs — all of which directly impact time-to-market and team output.

Research Gap & Objectives

Research

No existing IDE plugin unifies AI-powered CI/CD generation, architectural quality analysis, intelligent code generation, and runtime error diagnosis into a single tool — developers must context-switch between Spring Initializr, SonarQube, GitHub Actions, and separate LLM tools with no cross-module intelligence or audit trail.

Main

To design and deliver an IntelliJ IDEA plugin integrating four AI-driven modules — CI/CD Generation, Code Generation, Quality Analyzer, and Runtime Debugger — powered by AWS Bedrock and Google Gemini, reducing developer time on infrastructure and boilerplate tasks by at least 50%.

Solution Overview

Solution

An IntelliJ IDEA plugin that embeds four AI-powered modules directly into the developer's IDE, automating CI/CD generation, Spring Boot code generation, architectural quality analysis, and runtime error diagnosis using AWS Bedrock and Google Gemini.

Key features

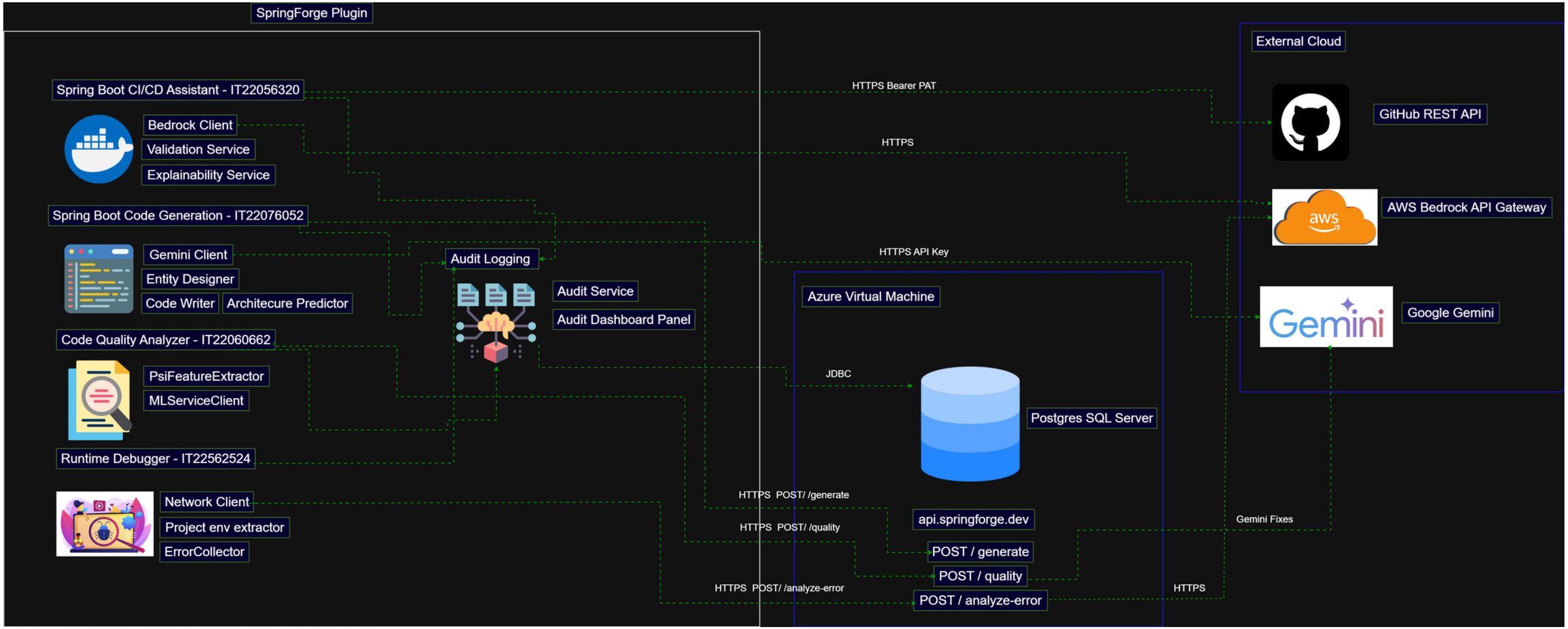
- Visual entity designer with Gemini-powered Java code generation and architecture detection (Code Generation)
- RAG-assisted Spring Boot runtime debugger with context-aware root cause analysis and fix suggestions. (Runtime Debugger)
- PSI-based architectural anti-pattern detection with AI fix suggestions via ML service (Quality Analyzer)
- AI-generated Dockerfile, Docker Compose, and GitHub Actions workflows with validation and explainability (CI/CD Generation)

Non-functional

- Usability
- Reliability
- Security
- Performance

Completion

System Architecture & Integration



Commercialization Strategy

Target users

- Spring Boot developers struggling with architecture decisions, code quality, and DevOps setup
- Junior Developers & Newcomers

Value

- Unlike standalone tools (SonarQube, GitHub Actions, Spring Initializr, ChatGPT), SpringForge is context-aware — it reads your actual project structure, dependencies, and code before generating anything, producing project-specific artifacts rather than generic templates, all without leaving the IDE.

Cost &

- Revenue Model: Freemium Subscription.
- Community: Free
- Professional and Enterprise editions with paid subscription with enhanced features

Adoption

- Pilot: Release Community edition to SLIIT student developers and open-source community to collect feedback on generation accuracy and usability
- Rollout: Publish to JetBrains Marketplace, promote via Spring Boot and Java developer communities to grow free user base
- Monetisation: Convert active Community users to Professional/Enterprise subscriptions through feature gating and team-level capabilities
- Future: Expand to Kotlin Multiplatform, GitLab CI, and team-level shared audit dashboards for Enterprise tier